Custom Image Classifier p1

This sic code is all you need to train a classifier with minor tweaks to preference and adjustments to data layers as needed.

1. Complete Source Code

```
from fastai.vision.all import *
from fastai.vision.widgets import *
import pandas as pd
import gradio as gr
# Step 1: Data Harmonization and Cleaning
folders_files = get_image_files('dataset_folder/')
folder_df = pd.DataFrame({
    "file": [str(x) for x in folder_files],
    "label": "Category_A"
})
#1 above we collected folder-labeled images into a DataFrame
# 2. load CSV-labeled dataset and prefix paths
csv_raw_df = pd.read_csv('dataset_csv/labels.csv')
csv_clean_df = csv_raw_df.dropna(subset=['file', 'label']).copy()
csv_clean_df['file'] = 'dataset_csv/' + csv_clean_df['file'].astype(str)
#3. merge into one master table
combined_df = pd.concat([folder_df, csv_clean_df[['file', 'label']]])
#4. Drop rows where path do not exist (if needed
...
#5 Delete/unlink corrupted images from disk if needed.
...
```

Custom Image Classifier p2

```
#Step 2: Datablock Blueprintt & Dataloaders
data_block = DataBlock(
    blocks = (ImageBlock, CategoryBlock),
    get_x = ColReader('file'),
    get_y = ColReader('label'),
    splitter = RandomSplitter(valid_pct=0.2, seed=42),
    item_tfms = Resize(224),
#   batch_tfms = aug_transforms()
)
    dls = data_block.dataloaders(combined_df, bs=64)
dls.show_batch(max_n=9)

#Step 3: Learner Intialization & Training
learn = vision_learner(dls, resnet34, metrics=error_rate)
learn.fine_tune(epochs=5)

#Step 4: Error Analysis
learn.show_results(max_n=6)

interp = ClassificationInterpretation.from_learner(learn)
interp.plot_confusion_matrix()
interp.plot_top_losses(k=5, nrows=5)
```

Custom Image Classifier p3

```
47  # Step 5: Model-Driven Cleaning
48  # requires folder-per-class layout (does NOT work with Col/DataFram labels)
49  cleaner = ImageClassifierCleaner(learn)
50  cleaner
51
52  # In seperate cell after selections from <ImageClassifierCleaner>
53  for idx in cleaner.delete():
54      cleaner.fns[idx].unlink()
55  for idx, cat in cleaner.change():
56      shutil.move(str(cleaner.fns[idx]), Path('dataset-folder') / cat)
57
58  # Step 6: Serilization & offline inference
59  learn.export('model-export.pkl')
60
61  learn-inf = load_learner('model-export.pkl')
62  pred-class, pred-idx, probs = learn-inf.predict('test-image.jpg')
63  pred-class
64
65  # Step 7: Gradio Web Application
66  categories = learn-inf.dls.vocab
67  def classify-image(img):
68      pred, pred-idx, probs = learn-inf.predict(img)
69      return dict(zip(categories, map(float, probs)))
70  interface = gr.Interface(
71    fn=classify-image, inputs=gr.Image(type='pil'), outputs=gr.Label())
72  interface.launch(inline=False)
```

Custom Image Classifier p4

Step 1: Data Harmonization

- Why a table at all: a classifier learns one function from image to label. One label space means one model, so every source has to be normalized into one uniform path-to-label mapping. DataFrame is where you do that merging in one place instead scattered.
- `str(x)` is required: `get_image_files` returns path objects, not strings. Pass them raw into `pd.DataFrame` and you get type error later.
- `'prefix/' + df['file']`: prepends a folder path to every row at once.
- `pd.concat([a, b], ignore_index=True)`: stacks two tables vertically and renumbers the rows.

Step 2: DataBlock Blueprint & DataLoaders

Defines how to read, split, and prepare the data, then builds the actual data feed.

- `blocks = (ImageBlock, CategoryBlock)`: input is an image, output is class label
- `get_x / get_y` with ColReader: where to find the image path and the label in the table.
- `RandomSplitter(valid_pct=0.2, seed=42)`: holds out 20% for validation; the seed makes the split repeatable.
- `item_tfms = Resize(224)`: makes every image the same size so they can be batched.
- `dataloaders(combined_df, bs=64)`: binds the blueprint to the table, 64 images per batch.
- `dls.show_batch(max_n=9)`: displays real image with label to confirm nothing is mismatched.

Custom Image Classifier p 5

Step 3: Learner Intializing & Training

Loads a pretrained network and adapts it to your classes.

- `vision_learner(dls, resnet34, metrics=error_rate)`: pairs the data with a pretrained resnet34 and reports error rate each epoch.
- `learn.fine_tune(epochs=5)`: trains the new output layer first, the unfreezes and trains the whole network.

Step 4: Error Analysis

Show where the model is wrong, not just how often.

- `learn.show_results(max_n=6)`: sample images with predicted vs actual labels.
- `ClassificationInterpretation.from_learner(learn)`: runs the validation set and stores predictions and losses.
- `interp.plot_confusion_matrix()`: grid of actual vs predicted, showing which classes get mixed up.
- `interp.plot_top_losses(k=5)`: the images the model got most wrong.

Step 5: Model-Driven Cleaning

Uses the trained model to surface bad data for manual review.

- `ImageClassificationCleaner(learn)`: jupyter widget showing high-loss images with a dropdown to delete or relabel each.
- `cleaner.delete()` / `cleaner.change()`: return the flagged indices; the widget records choices but changes nothing.
- `unlink()` and `shutil.move()`: the loops that actually delete and relabel files on disk.

Custom Image Classifier p6

Step 6: Serialization & Offline Inference

Saves the trained model and runs single predictions.

- `learn.export('model-export.pkl')`: writes architecture, weights, class names, and preprocessing into one file.
- `load_learner(...)`: reloads it, on CPU by default, with no data folders needed.
- `learn_inf.predict(img)`: one forward pass returning the label, its index, and the probabilities.

Step 7: Gradio Web Applications

Wraps the model in a web page.

- `classify_image(img)`: calls `predict` and pairs each class name with its probability.
- `gr.Interface(fn, inputs, outputs)`: connects the function to an upload box and a probability bar chart.
- `interface.launch(inline=False)`: starts the server at 127.0.0.1:7860

Chapter 2

01 Project Planning and Gathering

Deep Learning projects require framing real-world goals before writing code. Many teams fail by overcomplicating intial designs or waiting for perfect datasets. This lesson details framing objectives, collecting data and starting with a fast end-to-end baseline.

How should you frame a deep learning project?

- Drivetrain Approach guides the design of successful data products.
  - A Data product must start with a clear, actionable objective instead of model selection.
  - Levers define the specific decisions or actions you can control to achieve the objective.
  - Data requirements and predictive models must only be designed after defining objectives and levers.

How do we start and iterate on a project?

- Successful projects prioritize rapid, end-to-end iterations over localized perfection.
  - Modelers must complete a simple, working prototype from data to user interface as quickly as possible.
  - Early iterations can utilize shortcuts like running computations on remote servers with basic web interface.
  - Completing the entire pipeline immediatly exposes bottlenecks and reveals actual data needs.

Continued ch2

How do we gather and clean an image dataset?

- Image classification datasets can be gathered from internet search engines using simple download functions.
  - The download_images function downloads files directly to a destination path from a list of URL's
  - Web searches regularly return corrupt or invalid files that break training loops.
    - `verify_images` scans a list of files to identify items that cannot be opened as images.
    - `failed.map(Path.unlink)` deletes corrupt images from the filesystem automatically.

02 Building the Data Pipeline

Raw files on a disk cannot be fed directly into a neural network. Models require structured inputs and targets of uniform size. This lesson covers fastai's DataBlock API, splitters, resizing methods, and data augmentation.

What is the fastai DataBlock API?

- The datablock acts as a generic template to describe how to assemble raw data for training.
  - The DataBlock API requires five key pieces of information to construct a valid pipeline.
    - The `blocks` parameter defines type of independent x dependent vars
      - `ImageBlock` specifies that the inputs are image data.
      - `CategoryBlock` specifies that the targets are discrete categories.
    - `get_items` takes a dir path & returns list of raw items using `get_image_files`

Chp 2 continued

How do we split data and extract labels?

- A pipline must partition the data and associate each item with its correct ground-truth target.
  - `RandomSplitter` partitions data into training and validation sets randomly using a fixed seed for reproducible results.
  - The `valid_pct` parameter controls the proportion of data held out for validation, typically set to 0.2.
  - `get_y` defines the labeling function, using `parent_label` to extract the parent folder name as the label.

How should we resize images for mini-batches?

- Neural networks process multiple items in parallel using a tensor, which requires all images to have identical dimensions.
  - Item transforms apply resizing operations to individual images before grouping them into a batch.
  - not ideal **Resize** crops images by default to fit the requested dimensions, which risks losing critical peripheral details.
  - not ideal **Resize(method=ResizeMethod.Pad)** pads images with black borders, which wastes computation on empty space
  - not ideal **Resize(method=ResizeMethod.Squish)** stretches or squishes images, which distorts realistic shapes and reduces accuracy.

chpt 2 continued

Why do we use data augmentation?

- A neural network is initially entirely untrained and possesses no inherent understanding of spatial variations.
    - Data augmentation creates randomized visual variations of input data without altering its underlying category.
    - <Random Resize Crop> randomly crops different portions of each image on every epoch to teach the model spatial invariance.
    - <aug_transforms> provides a standard suite of augmentations including rotation, flipping, warping, and brightness adjustments.
    - <batch_tfms> applies augmentation to an entire batch simultaneously on the GPU, saving significant processing time.

```
0  bears = DataBlock(
1    blocks = (ImageBlock, CategoryBlock),
2    get_items = get_image_files,
3    splitter = RandomSplitter(valid_pct=0.2, seed=42),
4    get_y = parent_label,
5    item_tfms = Resize(224)
6  )
7  dls = bears.dataloaders(path)
```

- valid_pct, validation split. The % of dataset held out for eval (0.2)
    - High % reduces training but increases validation rediability.
    - Low % maximizes training data but risks a noisy validation estimate.
- seed, the random seed. Fixed starting point for random partition generation. Typically 42
    - Ensures identical train/validation splits across multiple runs to isolate model behavior.

chpt 2 continued

03 Model Training, Analysis, and Data Cleaning

Up to this point our data pipeline is fully constructed and prepared for modeling. Training a raw model often exposes mistakes in the training data itself. This lesson covers fitting the model, interpreting classification errors, and using model predictions to clean datasets.

How do we train the classifier?

- Training utilizes transfer learning to adapt pretrained vision models to custom datasets.
- The vision_learner function creates a convolutional neural network (CNN) using pretrained architectures like ResNet.
- Pretrained weights are loaded by default to leverage existing edge and feature detection capabilities.
- <fine_tune> updates model weights over a specified number of epochs, training the random head before fitting the rest of the network.

chpt 2 continued

How do we interpret model predictions?

- Model evaluation must occur exclusively on the validation set using intuitive metrics.
  - <Classification Interpretation> extracts predictions, targets, and losses to evaluate classification mistakes.
  - <plot_confusion_matrix> generates a grid comparing actual classes against predicted classes to identify specific categorical errors.
  - <plot_top_losses> sorts and displays validation images with the highest loss.
  - Loss rises when the model makes a confident incorrect prediction or a hesitant correct prediction.

How do we perform model-driven data cleaning?

- Cleaning a dataset is far faster and more accurate when guided by a trained model rather than manual pre-cleaning.
  - <Image Classifier Cleaner> provides an interactive GUI to identify and flag high-loss images for relabling or deletion.
  - The cleaner widget does not modify the filesystem directly but returns indices of selected files.
  - Relabeling relies on moving flagged files using shell commands like <shutil.move>
  - Deleting relies on unlinking flagged files using <Path.unlink>.

chpt 2 continued

```
learn = vision_learner(dls, resnet, metrics=error_rate)
learn.fine_tune(4)


interp = ClassificationInterpretation.from_learner(learn)
interp.plot_confusion_matrix()
interp.plot_top_losses(5, nrows=1)


cleaner = ImageClassifierCleaner(learn)
for idx in cleaner.delete(): cleaner.fns[idx].unlink()
for idx, cat in cleaner.change(): shutil.move(str(cleaner.fns[idx]
), path/cat)
```

- **resnet18 vs resnet34**, the network depth.

choose variant of ResNet architecture by number of layers

- more layers improve accuracy on large datasets but increase training time and risk overfitting.
- fewer layers train faster and require less GPU memory but have lower learning capacity.

04 App Development and Deployment

A trained model must be saved and wrapped in an application to be useful. Data scientists must design interfaces and deploy them cost-effectively to production environments. This lesson covers model exporting, inference, prototyping with Gradio, and deployment infrastructure.

How do we save and load a model for inference?

- Deploying a model requires saving both it's mathamatical architecture and trained parameters.
  - The export method serializes the complete model and data transforms into a single `<export.pkl>` file. (this is serialized file)
    - `<load_learner>` reconstructs the working model from the serialized file on the production server.
- **Inference** is the process of getting predictions for a single new item using `<learn_inf.predict>`
  - `<predict>` returns three outputs: the predicted category string, it's index, and the probabilities for all classes.

How do we build an interactive prototype with Gradio?

- Data products require interactive user interfaces to capture inputs and display predictions.
  - Gradio provides a rapid python-only framework to wrap ML models in iterative web pages.
    - Prediction func. takes raw image, calls `<learn.predict>`, returns {category name: ### prob}
    - `<gr.Interface>` links the predictive func to inputs `gr.Image()` and outputs
    - `<launch()>` hosts the application locally or generates a public sharing img

to gen UI

chpt 2... continued

Where should we host app?

- Gradio UI on hugging face spaces (paid).

05 Prevent failures in Production

Deployed models are only one component of a much larger, dynamic system. Models are brittle and can fail catastrophically when real world distributions shift. Lesson covers addressing.

What are the main systematic failure modes?

- DL models are highly flexible but analytically impossible to fully understand, leading to unpredictable behaviors.
  - Out-of-domain data occurs when production inputs differ significantly in structure or style from the training data.
  - Domain shift represents a gradual change in real world data over time, making original training data irrelevent.
  - A bear detector trained on clean internet photos will fail on low-resolution, nighttime video frames.

How do we mitigate deployment risks?

- Deploying models directly to automated actions creates severe organizational and societal risks
  - Model rollouts must follow a strictly supervised, multi-stage rollout process to catch failures early.
  - Stage 1 runs the model in parrallel with a manual process, using outputs only for human inspection and logging.
  - Stage 2 limits the model to small, highly supervised trial
  - Stage 3 expands the rollout slowly while establishing robust reporting to monitor anomalies.

chapt 2 — continued

What is a feedback loop?

- Deployed models actively interact with and alter the behavior of the environments they operate in.
  - A feedback loop occurs when model predictions reinforces the biases in the data generation process, creating a self-fulfilling cycle.
  - Predictive policing models send officers to high-arrest areas, leading to more arrests there, which further biases the model.
  - Recomendation algos favor popular items, encouraging more views, which drives the model to recommend those items exclusively.